

SLIDE 04 – Search & Filter

Objective

Quản lý trạng thái ứng dụng trong React và xây dựng danh mục sản phẩm có thể tương tác.

Theory

1. React Hooks

Trong React, `useState` dùng để lưu trạng thái như từ khóa tìm kiếm, danh mục đang chọn, kiểu sắp xếp. `useEffect` thường dùng để gọi API khi component được render lần đầu. Với danh sách sản phẩm, React thường dùng `filter()` và `map()` để lọc dữ liệu rồi hiển thị ra giao diện.

2. Data Handling & Axios GET

Axios được dùng để gửi request đến API. Khi gọi `axios.get()`, dữ liệu trả về thường nằm trong `response.data`, sau đó ta lưu vào state để render lên giao diện.

3. Layout

Có thể thiết kế giao diện bằng CSS thường, Bootstrap, Tailwind hoặc CSS Modules. Trong bài lab này dùng CSS thường để dễ hiểu và dễ chỉnh sửa.

Hands-on Practice

Cài đặt các chức năng:

- Tìm kiếm sản phẩm theo tên.
- Lọc sản phẩm theo danh mục.
- Sắp xếp sản phẩm theo giá tăng dần hoặc giảm dần.

Outcome

Sau bài lab, ứng dụng có **interactive product catalog**, người dùng có thể tìm kiếm, lọc và sắp xếp sản phẩm trực tiếp trên giao diện.

LAB 04 – Product Search, Category Filter, Price Sorting

1. Cài Axios

Trong thư mục frontend, chạy:

```
npm install axios
```

2. Tạo file `src/api.js`

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://127.0.0.1:8000",
});

export const getProducts = async () => {
  const response = await api.get("/products");
  return response.data;
};
```

File này dùng để gọi API lấy danh sách sản phẩm từ backend FastAPI.

3. Tạo file `src/components/ProductCatalog.jsx`

```
import { useEffect, useMemo, useState } from "react";
import { getProducts } from "../api";
import "../ProductCatalog.css";

function ProductCatalog() {
  const [products, setProducts] = useState([]);
  const [searchText, setSearchText] = useState("");
  const [selectedCategory, setSelectedCategory] = useState("all");
  const [sortType, setSortType] = useState("default");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const loadProducts = async () => {
      try {
        const data = await getProducts();
        setProducts(data);
      } catch (error) {
        console.error("Lỗi khi lấy sản phẩm:", error);
      } finally {
        setLoading(false);
      }
    };

    loadProducts();
  }, []);

  const categories = useMemo(() => {
    const list = products.map((product) => product.category || "Khác");
```

```

    return ["all", ...new Set(list)];
  }, [products]);

const filteredProducts = useMemo(() => {
  let result = [...products];

  if (searchText.trim() !== "") {
    result = result.filter((product) =>
      product.name.toLowerCase().includes(searchText.toLowerCase())
    );
  }

  if (selectedCategory !== "all") {
    result = result.filter(
      (product) => product.category === selectedCategory
    );
  }

  if (sortType === "price-asc") {
    result.sort((a, b) => a.price - b.price);
  }

  if (sortType === "price-desc") {
    result.sort((a, b) => b.price - a.price);
  }

  return result;
}, [products, searchText, selectedCategory, sortType]);

if (loading) {
  return <p>Đang tải sản phẩm...</p>;
}

return (
  <div className="catalog-container">
    <h2>Product Catalog</h2>

    <div className="filter-bar">
      <input
        type="text"
        placeholder="Tìm kiếm sản phẩm..."
        value={searchText}
        onChange={(e) => setSearchText(e.target.value)}
      />

      <select
        value={selectedCategory}
        onChange={(e) => setSelectedCategory(e.target.value)}
      >
        {categories.map((category) => (
          <option key={category} value={category}>
            {category === "all" ? "Tất cả danh mục" : category}
          </option>
        ))}
      </select>

      <select value={sortType} onChange={(e) =>
        setSortType(e.target.value)}>
        <option value="default">Sắp xếp mặc định</option>
        <option value="price-asc">Giá tăng dần</option>
        <option value="price-desc">Giá giảm dần</option>
      </select>
    </div>
  </div>

```

```

    <p className="result-count">
      Tìm thấy {filteredProducts.length} sản phẩm
    </p>

    <div className="product-grid">
      {filteredProducts.length > 0 ? (
        filteredProducts.map((product) => (
          <div className="product-card" key={product.id}>
            <h3>{product.name}</h3>
            <p>Danh mục: {product.category}</p>
            <p className="price">{product.price.toLocaleString()} VNĐ</p>
          </div>
        ))
      ) : (
        <p>Không tìm thấy sản phẩm phù hợp.</p>
      )}
    </div>
  </div>
);
}

export default ProductCatalog;

```

`useMemo` phù hợp ở đây vì danh sách đã lọc/sắp xếp chỉ cần tính lại khi `products`, `searchText`, `selectedCategory` hoặc `sortType` thay đổi. React mô tả `useMemo` là hook dùng để cache kết quả tính toán giữa các lần render.

4. Tạo file `src/components/ProductCatalog.css`

```

.catalog-container {
  padding: 30px;
}

.catalog-container h2 {
  margin-bottom: 20px;
}

.filter-bar {
  display: flex;
  gap: 12px;
  margin-bottom: 20px;
  flex-wrap: wrap;
}

.filter-bar input,
.filter-bar select {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 6px;
  min-width: 200px;
}

.result-count {
  margin-bottom: 16px;
  font-weight: 500;
}

.product-grid {
  display: grid;

```

```

    grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
    gap: 18px;
}

.product-card {
  padding: 18px;
  border: 1px solid #ddd;
  border-radius: 10px;
  background-color: white;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

.product-card h3 {
  margin-bottom: 10px;
}

.price {
  font-weight: bold;
  color: #1976d2;
}

```

5. Sửa file `src/App.jsx`

```

import ProductCatalog from "../components/ProductCatalog";

function App() {
  return (
    <div>
      <ProductCatalog />
    </div>
  );
}

export default App;

```

6. Dữ liệu API cần có dạng như sau

Backend `/products` nên trả về dữ liệu kiểu:

```

[
  {
    "id": 1,
    "name": "Laptop Dell",
    "category": "Laptop",
    "price": 15000000
  },
  {
    "id": 2,
    "name": "iPhone 15",
    "category": "Điện thoại",
    "price": 22000000
  },
  {
    "id": 3,
    "name": "Tai nghe Bluetooth",
    "category": "Phụ kiện",
    "price": 500000
  }
]

```

```
}  
]
```

Expected Result

Sau khi hoàn thành Lab 04, giao diện sẽ có:

- Ô tìm kiếm sản phẩm.
- Bộ lọc danh mục.
- Bộ sắp xếp giá.
- Danh sách sản phẩm tự động thay đổi theo điều kiện người dùng chọn.

Ví dụ:

Người dùng nhập **“iphone”** → chỉ hiện sản phẩm iPhone.

Người dùng chọn **“Laptop”** → chỉ hiện sản phẩm thuộc danh mục Laptop.

Người dùng chọn **“Giá giảm dần”** → sản phẩm được sắp xếp từ giá cao xuống thấp.



